Ingeniería del Software

Educación Media Superior

Gestión, planificación y control de proyectos

Mag. Lic. Eduardo Olivera

Las 4 P de la Gestión de Proyectos de Software

(Basado en Pressman – Ingeniería de Software.

Adaptado del material del Prof. Marcelo Oliver)

Introducción: La Gestión como Pilar del Éxito en Software

Cuando nos adentramos en el mundo de la ingeniería de software, la imagen mental predominante suele ser la del programador resolviendo problemas complejos, diseñando arquitecturas elegantes o realizando pruebas exhaustivas.

Las 4 P de la Gestión de Proyectos de Software

Introducción

Si bien estas actividades técnicas son el núcleo de la construcción, Roger Pressman, en su obra seminal "Ingeniería del Software: Un Enfoque Práctico", plantea una premisa fundamental que redefine las prioridades:

"La mayoría de los fracasos en proyectos de software no se deben a errores de programación, sino a fallas de gestión."

Las 4 P de la Gestión de Proyectos de Software

Introducción

Gestionar un proyecto de software es mucho más que "organizar tareas" en una lista.

Es una disciplina compleja que implica coordinar un ecosistema dinámico compuesto por:

• Personas: Con sus habilidades, motivaciones, conflictos y necesidades de comunicación.

• Objetivos: A menudo cambiantes, ambiguos o contradictorios entre los stakeholders.

• Restricciones: De tiempo, presupuesto, tecnología, recursos y normativas legales.

• Incertidumbre: Asociada a la tecnología, a la estimación y a los riesgos no identificados.

• Cambio: La única constante en el software, que debe ser gestionado para que no se convierta en caos.s

Las 4 P de la Gestión de Proyectos de Software

Para navegar esta complejidad, Pressman propone un modelo conceptual simple pero profundamente poderoso: Las 4 P de la Gestión de Proyectos de Software.

Este marco no es una lista de verificación estática, sino un sistema de dimensiones interconectadas que, analizadas en conjunto, proporcionan una visión holística del proyecto.

Las 4 P de la Gestión de Proyectos de Software

Las 4 P son:

1. People (Personas)
2. Product (Producto)
3. Process (Proceso)
4. Project (Proyecto)

El orden en que se presentan no es casual. Establece una jerarquía de dependencia, donde el factor humano es la base sobre la que se construye todo lo demás.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS: El Corazón del Proyecto

Idea Central: El Factor Humano como Activo Primordial

El software es, ante todo, un producto intelectual. Es el resultado del pensamiento, la creatividad y el esfuerzo de personas.

Ninguna herramienta, metodología o tecnología por sí sola puede construir software de valor.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Pressman enfatiza con vehemencia: "Si el equipo no funciona correctamente, el proyecto no funcionará, aunque el proceso sea excelente".

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

¿Qué implica gestionar personas?

Va mucho más allá de la asignación de tareas. Es una función de liderazgo y desarrollo humano que abarca:

- Formación de Equipos Equilibrados
- Selección de Perfiles Adecuados
- Fomento de la Comunicación
- Resolución de Conflictos
- Motivación
- Mantenimiento de la Estabilidad

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Competencia Técnica y Estratégica

No todos los desarrolladores poseen el mismo nivel de habilidad ni las mismas fortalezas.

Algunos son especialistas en front-end, otros en bases de datos, algunos son pensadores abstractos ideales para la arquitectura, y otros son meticulosos y perfectos para el aseguramiento de la calidad.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Comunicación: El Tejido Conectivo del Equipo

La comunicación es el lubricante que permite que el motor del equipo funcione sin fricción. Su complejidad crece de manera exponencial con el tamaño del equipo.

La fórmula [n(n-1)]/2* (donde 'n' es el número de personas) nos da el número de canales de comunicación potenciales.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Comunicación

Con 4 personas: 6 canales.

Con 8 personas: 28 canales.

La comunicación informal colapsa cuando el equipo crece.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Conclusión Clave

Añadir más personas a un proyecto que está retrasado no siempre acelera las cosas.

De hecho, puede empeorarlas al aumentar la carga de comunicación y el tiempo de integración.

Esto se relaciona con la conocida Ley de Brooks.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Liderazgo

El líder de proyecto no es simplemente un "programador senior" con más responsabilidades.

Es un rol organizacional distinto, cuyo objetivo es guiar al equipo hacia el éxito.

Su enfoque principal son las personas y el proyecto, no el código.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Responsabilidades del líder

- Toma de decisiones estratégicas
- Mediación de conflictos
- Definición de prioridades
- Gestión de la presión externa
- Mantenimiento de la cohesión del equipo

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Estabilidad del equipo

La rotación de personal es uno de los riesgos más graves y costosos en el desarrollo de software.

Cuando un miembro clave del equipo abandona el proyecto, el impacto es multidimensional.

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Impactos de la rotación

- Pérdida de conocimiento tácito
- Retraso en la integración de nuevos miembros
- Aumento del riesgo de errores

Las 4 P de la Gestión de Proyectos de Software

PEOPLE – PERSONAS

Conclusión sobre PEOPLE

El software es un producto intelectual colectivo.

Descuidar la gestión humana no es solo un riesgo; es garantizar la erosión de los cimientos del proyecto.

Por ello, las personas constituyen el primer pilar y el más fundamental de las 4 P.

Las 4 P de la Gestión de Proyectos de Software

PRODUCT – PRODUCTO: Definiendo el "Qué"

Antes de cualquier planificación, estimación o asignación de recursos, debemos responder a la pregunta fundamental:

¿Qué vamos a construir?

Pressman lo plantea con claridad:

"No se puede estimar ni planificar lo que no se comprende".

Las 4 P de la Gestión de Proyectos de Software

PRODUCTO

Definir el PRODUCTO implica Establecer de manera rigurosa:

- Alcance
- Objetivos
- Funcionalidades
- Requerimientos no funcionales
- Restricciones
- Contexto organizacional
- Factibilidad

Las 4 P de la Gestión de Proyectos de Software

PRODUCTO

Alcance

El alcance es la frontera que separa lo que el sistema hará de lo que no hará.

Es un contrato implícito entre el equipo de desarrollo y el cliente.

Las 4 P de la Gestión de Proyectos de Software

PRODUCTO

Scope Creep

Si el alcance no se define correctamente aparece el fenómeno llamado scope creep, donde el sistema comienza a crecer sin control.

Esto termina desbordando el presupuesto y el cronograma.

Ejemplo: El sistema a desarrollar permitirá registrar libros, registrar usuarios y gestionar préstamos y devoluciones, además de consultar la disponibilidad de libros.

El sistema será utilizado por el personal de la biblioteca dentro de la institución.

El sistema no incluirá la compra de libros, la lectura de libros digitales, la gestión de multas ni el acceso desde internet.

Las 4 P de la Gestión de Proyectos de Software

Objetivos

Los objetivos del producto describen los resultados que el sistema debe lograr para satisfacer las necesidades del usuario o de la organización. Indican el propósito del sistema y el valor que aportará. (Pressman)

Objetivo general → propósito principal del sistema

Objetivos específicos → resultados concretos que el sistema debe cumplir

Las 4 P de la Gestión de Proyectos de Software

Objetivos

Ejemplo: Sistema de gestión de biblioteca.

Objetivo general: Gestionar el préstamo y devolución de libros.

Objetivos específicos:

- Registrar usuarios.
- Registrar préstamos.
- Controlar multas por atraso.
- Generar listados de libros disponibles.

Las 4 P de la Gestión de Proyectos de Software

Requerimientos funcionales y no funcionales

Mientras los requerimientos funcionales describen qué hace el sistema (ingreso de cliente, eliminar un cliente, etc), los no funcionales describen cómo lo hace (rendimiento, seguridad, disponibilidad, mantenibilidad, escalabilidad, etc)

Las 4 P de la Gestión de Proyectos de Software

Restricciones

Las restricciones son limitaciones que afectan el desarrollo o funcionamiento del sistema, impuestas por factores técnicos, económicos, legales u organizacionales. (Sommerville)

Ejemplos:

- El sistema debe desarrollarse en Java.
- Debe utilizar MySQL como base de datos.
- Debe ejecutarse en computadoras existentes de la empresa.
- El sistema debe estar terminado en 4 meses.
- Debe cumplir la normativa de protección de datos.

Las 4 P de la Gestión de Proyectos de Software

Contexto organizacional

El contexto organizacional describe el entorno en el cual el sistema será utilizado, incluyendo la estructura de la organización, los usuarios, procesos de negocio y sistemas existentes con los que interactúa. (Pressman / Sommerville)

Ejemplo: Sistema para una clínica:

- Usuarios: recepcionistas, médicos y administradores.
- El sistema se usará en la red interna de la clínica.
- Debe integrarse con el sistema contable existente.
- El sistema se utilizará durante el horario de atención.
- La clínica tiene 5 consultorios y 10 computadoras.

Las 4 P de la Gestión de Proyectos de Software

Factibilidad

Antes de comprometer recursos significativos, es necesario realizar un estudio de factibilidad.

Si es factible, se aprueba el proyecto.

La factibilidad se estudia antes de iniciar el proyecto, para decidir si el producto debe desarrollarse.

Las 4 P de la Gestión de Proyectos de Software

Factibilidad

Tipos:

Tipo	Pregunta que responde
Técnica	¿Se puede hacer con la tecnología disponible?
Económica	¿Es rentable / hay presupuesto?
Temporal	¿Se puede hacer en el tiempo requerido?
Operativa	¿La organización podrá usar el sistema?

Las 4 P de la Gestión de Proyectos de Software

PRODUCTO

Conclusión

Definir mal el producto es sembrar el fracaso desde la génesis del proyecto.

Una comprensión superficial del "qué" lleva a estimaciones erróneas, planificaciones irreales y productos que no satisfacen las necesidades del usuario.

Las 4 P de la Gestión de Proyectos de Software

PROCESS – PROCESO: El Camino a Seguir

Si el Producto define el "qué", el Proceso define el "cómo".

Es el marco estructurado de actividades, acciones y tareas que guían el trabajo del equipo para construir el software.

Las 4 P de la Gestión de Proyectos de Software

PROCESO

Proceso vs Proyecto

Proceso: conjunto organizado de actividades, principios y prácticas para desarrollar software.

Proyecto: instancia concreta que ejecuta ese proceso para construir un producto específico.

Las 4 P de la Gestión de Proyectos de Software

PROCESO

Beneficios del proceso

- Reduce la improvisación
- Permite el control y la visibilidad
- Define entregables
- Establece validaciones

Las 4 P de la Gestión de Proyectos de Software

PROCESO

Modelos de proceso

Pressman clasifica los modelos de proceso en:

- Modelo en Cascada
- Modelo Incremental
- Modelos Evolutivos
- Modelo en Espiral

Las 4 P de la Gestión de Proyectos de Software

PROJECT – PROYECTO: La Gestión en Ejecución

Gestionar el proyecto es el arte de aplicar conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para cumplir con sus requisitos.

Las 4 P de la Gestión de Proyectos de Software

PROYECTO

Funciones clásicas de la gestión:

- Planificación
- Estimación
- Gestión de riesgos
- Seguimiento y control

Las 4 P de la Gestión de Proyectos de Software

PROYECTO

Planificación

La planificación consiste en descomponer el trabajo futuro en unidades manejables y ordenarlas en el tiempo.

Las 4 P de la Gestión de Proyectos de Software

PROYECTO

Estimación

Estimar es el proceso de predecir el esfuerzo, el tiempo y el costo necesarios para completar las tareas del proyecto.

Gestión de riesgos

La gestión de riesgos es un proceso proactivo y continuo para identificar, analizar y monitorear riesgos que pueden amenazar el éxito del proyecto.

Las 4 P de la Gestión de Proyectos de Software

PROYECTO

Seguimiento y control

Un proyecto requiere monitoreo constante para comparar el progreso real con el planificado y tomar medidas correctivas.

Las 4 P de la Gestión de Proyectos de Software

Interrelación de las 4P

Las 4 P no son compartimentos estancos.

Funcionan como un sistema dinámico donde un cambio en una de ellas provoca un efecto en las demás.

Las 4 P de la Gestión de Proyectos de Software

La gestión de proyectos de software no es una tarea administrativa secundaria.

Es la columna vertebral que permite transformar una idea abstracta en un sistema de software real, funcional y valioso.

Actividad práctica: ejercicio 3.1

Gestión de Proyectos de Software

Desarrollar software no consiste únicamente en programar.

Un proyecto de software implica:

- coordinar personas
- organizar tareas
- gestionar recursos
- cumplir plazos
- asegurar la calidad del producto

Por lo tanto, el desarrollo de software requiere **gestión del proyecto**.

La gestión del proyecto permite transformar una idea en un sistema funcional.

Gestión de Proyectos de Software

¿Qué es un proyecto de software?

Un proyecto de software es un esfuerzo temporal cuyo objetivo es crear un sistema informático que resuelva un problema específico.

Todo proyecto se caracteriza por:

- tener un **objetivo definido**
- tener **recursos limitados**
- desarrollarse en un **tiempo determinado**
- producir un **resultado concreto**

En ingeniería de software, ese resultado es un **producto de software**.

Gestión de Proyectos de Software

Características de los proyectos de software

Los proyectos de software tienen características particulares:

- alta **complejidad**
- requisitos que pueden **cambiar**
- fuerte dependencia del **equipo humano**
- necesidad de **coordinación constante**

Por estas razones, los proyectos deben planificarse y controlarse cuidadosamente.

Gestión de Proyectos de Software

¿Por qué es necesaria la gestión del proyecto?

Sin una gestión adecuada pueden aparecer problemas como:

- retrasos en el desarrollo
- falta de coordinación entre integrantes
- funcionalidades incompletas
- mala calidad del software
- aumento de costos

La gestión del proyecto busca **reducir estos riesgos**.

Gestión de Proyectos de Software

Actividades principales en la gestión de proyectos

La gestión de proyectos de software incluye varias actividades:

- planificación del proyecto
- organización del equipo
- estimación de tiempos y recursos
- seguimiento del avance
- gestión de cambios
- control de calidad

Estas actividades se desarrollan durante todo el ciclo de vida del proyecto.

Gestión de Proyectos de Software

Planificación del proyecto

Planificar un proyecto significa:

- definir **qué tareas se realizarán**
- determinar **quién las realizará**
- establecer **cuándo se realizarán**
- identificar **dependencias entre tareas**

La planificación permite organizar el trabajo del equipo.

Gestión de Proyectos de Software

Descomposición del proyecto en tareas

Un proyecto grande debe dividirse en **tareas más pequeñas**.

Esto permite:

- comprender mejor el trabajo a realizar
- asignar responsabilidades
- estimar tiempos de manera más realista
- controlar el progreso del proyecto

Este proceso se conoce como **descomposición del trabajo**.

Gestión de Proyectos de Software

Hitos del proyecto

Un **hito** es un punto importante dentro del proyecto.

Representa la finalización de una etapa relevante.

Ejemplos de hitos en un proyecto de software:

- finalización de requerimientos
- finalización del diseño
- primera versión funcional del sistema
- finalización de pruebas

Los hitos permiten evaluar el avance del proyecto.

Gestión de Proyectos de Software

Seguimiento del proyecto

La gestión del proyecto no termina con la planificación.

Durante el desarrollo es necesario:

- controlar el avance del trabajo
- detectar retrasos
- resolver problemas
- realizar ajustes en la planificación

Este proceso se denomina **seguimiento y control del proyecto**.

Gestión de Proyectos de Software

Gestión de proyectos y trabajo en equipo

Un proyecto de software se desarrolla **en equipo**.

Por ello es importante:

- definir **roles**
- asignar **responsabilidades**
- mantener una **comunicación clara**
- coordinar el trabajo entre los integrantes

La organización del equipo es un factor clave para el éxito del proyecto.

Actividad práctica: ejercicio 3.2

Planificación del proyecto

Diagrama de Gantt

En un proyecto de software es necesario organizar el trabajo.

Para ello debemos definir:

- qué tareas se realizarán
- cuándo se realizarán
- cuánto tiempo durarán
- en qué orden se ejecutarán

Una herramienta muy utilizada para representar esta planificación es el diagrama de Gantt.

Planificación del proyecto

Diagrama de Gantt

¿Qué es un diagrama de Gantt?

Un diagrama de Gantt es una representación gráfica de las tareas de un proyecto a lo largo del tiempo.

Fue desarrollado por Henry Gantt a principios del siglo XX.

Permite visualizar: las tareas del proyecto, la duración de cada tarea, el orden en que se realizan, el avance del proyecto

Planificación del proyecto

Diagrama de Gantt

Elementos de un diagrama de Gantt.

Un diagrama de Gantt se compone de:

Tareas: Actividades necesarias para completar el proyecto.

Duración: Tiempo que lleva completar cada tarea.

Cronograma: Línea temporal del proyecto.

Barras de tareas: Representan gráficamente la duración de cada tarea.

Planificación del proyecto

Diagrama de Gantt

Ejemplo:

Proyecto: desarrollo de un sistema de biblioteca.

Tareas del proyecto: analizar requerimientos, diseñar la base de datos

programar el sistema, realizar pruebas

Cada tarea tiene una duración estimada.

Planificación del proyecto

Diagrama de Gantt

Ejemplo de planificación:

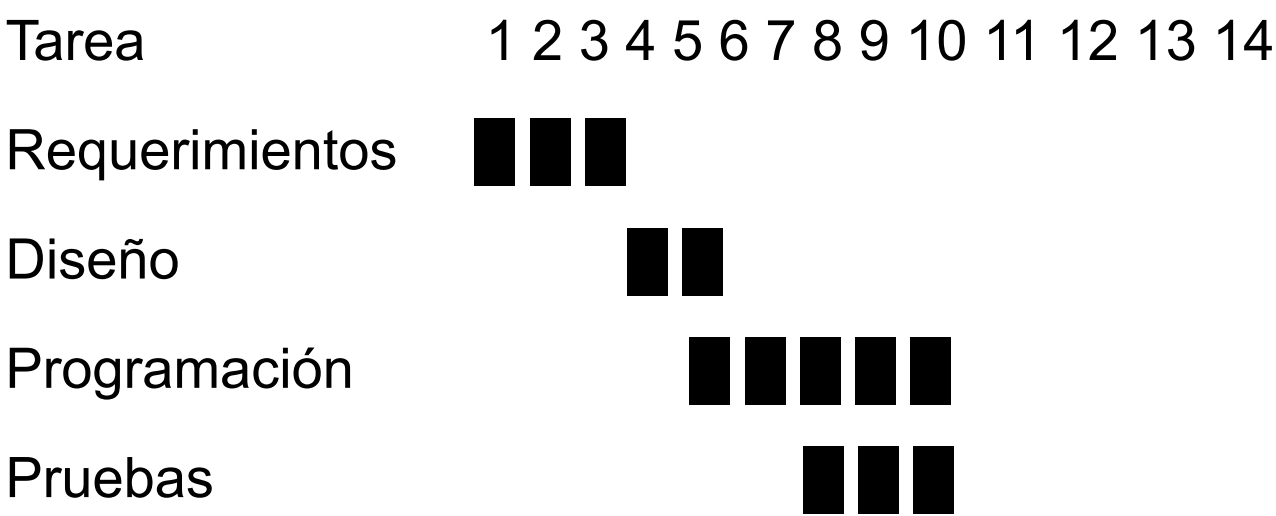
Tarea	Duración
Requerimientos	3 días
Diseño	2 días
Programación	6 días
Pruebas	3 días

Esto representa la **planificación básica del proyecto**.

Planificación del proyecto

Diagrama de Gantt

Representación en Gantt - Ejemplo (cada barra representa el tiempo que ocupa una tarea):



Planificación del proyecto

Diagrama de Gantt

Ventajas del diagrama de Gantt:

- visualizar todo el proyecto
- entender el orden de las tareas
- controlar el avance del trabajo
- detectar retrasos
- organizar el trabajo del equipo

Por esta razón es una de las herramientas más utilizadas en gestión de proyectos.

Planificación del proyecto

Diagrama de Gantt

Uso del Gantt en proyectos de software:

Se utiliza para planificar etapas como: análisis de requerimientos, diseño del sistema, desarrollo del software, pruebas, entrega final

Esto permite coordinar el trabajo del equipo.

Planificación del proyecto

Diagrama de Gantt

Ejemplo aplicado a un proyecto estudiantil. Proyecto: sistema de reservas de canchas deportivas. Posible planificación:

Tarea	Duración
Requerimientos	2 días
Diseño	2 días
Programación	5 días
Pruebas	2 días

Planificación del proyecto

Diagrama de Gantt

Representación en Gantt:

Tarea	1	2	3	4	5	6	7	8	9
Requerimientos	■	■							
Diseño		■	■						
Programación			■	■	■	■	■		
Pruebas								■	■

Esto muestra el cronograma del proyecto.

Planificación del proyecto

Diagrama de Gantt

Limitaciones del diagrama de Gantt:

Aunque es una herramienta muy útil, el diagrama de Gantt tiene algunas limitaciones: no muestra claramente las dependencias entre tareas, puede volverse complejo en proyectos grandes, requiere actualizarse constantemente

Aun así, es una herramienta fundamental en la planificación de proyectos.

Planificación del proyecto

Diagrama de Gantt

Conclusión

El diagrama de Gantt permite:

- planificar el trabajo del proyecto
- visualizar el tiempo de cada tarea
- coordinar el trabajo del equipo

Por ello es una herramienta clave en la gestión de proyectos de software.

Actividad práctica: ejercicio 3.3

Estructura de red de tareas (PERT)

Hasta ahora vimos cómo organizar tareas en el tiempo usando Gantt.

Ahora veremos cómo se relacionan las tareas entre sí dentro de un proyecto.

¿Qué es una red de tareas?

Una red de tareas es una representación del proyecto que muestra:

- las tareas
- el orden en que deben realizarse
- las dependencias entre ellas

Permite entender la estructura lógica del proyecto.

Estructura de red de tareas (PERT)

Diferencia con Gantt

El diagrama de Gantt muestra: cuándo se hacen las tareas

La red de tareas muestra: en qué orden deben hacerse

Ambos se complementan.

Estructura de red de tareas (PERT)

Dependencias entre tareas

Una tarea puede depender de otra.

Ejemplo: no se puede programar sin antes definir los requerimientos

Esto se llama dependencia.

Estructura de red de tareas (PERT)

Tipos de relación

- una tarea comienza cuando termina otra

Ejemplo:

Requerimientos → Diseño → Programación → Pruebas

Estructura de red de tareas (PERT)

Representación de una red

Las tareas se representan como bloques conectados por flechas.

Ejemplo:

Requerimientos → Diseño → Programación → Pruebas

Las flechas indican el orden del proyecto.

Estructura de red de tareas (PERT)

Ejemplo con tareas paralelas

Algunas tareas pueden hacerse al mismo tiempo.

Ejemplo. Requerimientos:

→ Diseño de interfaz

→ Diseño de base de datos

→ Programación

Aquí hay trabajo en paralelo.

Estructura de red de tareas (PERT)

¿Por qué es importante?

La red de tareas permite:

- entender el flujo del proyecto
- detectar tareas críticas
- identificar trabajo en paralelo
- mejorar la planificación

Estructura de red de tareas (PERT)

Introducción a la ruta crítica

Dentro de todas las tareas, algunas son más importantes que otras.

La ruta crítica es: la secuencia de tareas que determina la duración total del proyecto.

Idea clave de ruta crítica: Si una tarea de la ruta crítica se retrasa,

todo el proyecto se retrasa.

Son las tareas que no pueden demorarse.

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Paso 1: identificar todas las relaciones (caminos posibles)

Paso 2: sumar la duración de cada camino

Paso 3: el camino más largo es la ruta crítica

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

¿Cómo se determinan las relaciones?

Se obtienen a partir de:

- el análisis del trabajo
- el orden lógico de construcción del sistema
- las dependencias reales entre tareas

Pregunta clave: ¿Qué tarea necesita que otra termine para poder comenzar?

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Ejemplo: Proyecto: sistema de biblioteca

Tareas:

Tarea	Descripción
A	Requerimientos
B	Diseño
C	Programación
D	Pruebas

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Determinación de dependencias:

Analizamos cada tarea:

- ¿Se puede diseñar sin requerimientos? → ❌
- ¿Se puede programar sin diseño? → ❌
- ¿Se puede probar sin programar? → ❌

Entonces: **A → B → C → D**

Este es un flujo **secuencial**.

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Ejemplo con tareas en paralelo: Proyecto: sistema web

Tareas:

Tarea	Descripción
A	Requerimientos
B	Diseño interfaz
C	Diseño base de datos
D	Programación

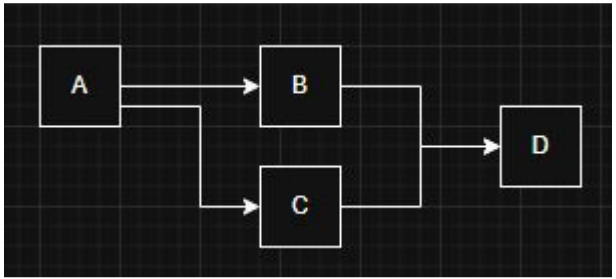
Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Análisis de dependencias:

- B y C dependen de A
- B y C pueden hacerse en paralelo
- D depende de B y C

Representación de la red::



Esto representa:

- trabajo en paralelo
- convergencia en una tarea final

Aquí aparecen **dos caminos posibles** (A-B-D y A-C-D)

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Agregamos la duración de cada tarea:

Tarea	Duración
A	2 días
B	3 días
C	4 días
D	2 días

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Identificación de caminos

Camino 1: A → B → D

2 + 3 + 2 = 7 días

Camino 2: A → C → D

2 + 4 + 2 = 8 días

Ruta crítica: A → C → D

Porque es el camino más largo.

Duración total del proyecto: 8 días

Estructura de red de tareas (PERT)

Cómo identificar la ruta crítica

Interpretación:

Las tareas críticas son: A, C y D.

Si alguna se retrasa: se retrasa todo el proyecto.

Estructura de red de tareas (PERT)

Introducción a la holgura

Hasta ahora vimos que:

- la ruta crítica es el camino más largo
- si una tarea crítica se retrasa, se retrasa todo el proyecto

Pero…

¿qué pasa con las tareas que NO son críticas?

Estructura de red de tareas (PERT)

Introducción a la holgura

La holgura es:

el tiempo que una tarea puede retrasarse sin afectar la duración total del proyecto.

Las tareas que no están en la ruta crítica tienen holgura.

Estructura de red de tareas (PERT)

Introducción a la holgura

Ejemplo con tres caminos - Tareas y duraciones:

A = 2 días

B = 3 días

C = 4 días

E = 2 días

D = 2 días

Relaciones:

A → B → D
A → C → D
A → E → D

Estructura de red de tareas (PERT)

Introducción a la holgura

Identificación de caminos:

Camino 1: A → B → D

2 + 3 + 2 = 7 días

Camino 2: A → C → D

2 + 4 + 2 = 8 días

Camino 3: A → E → D

2 + 2 + 2 = 6 días

Ruta crítica: A → C → D

Porque es el camino más largo.

Duración total del proyecto: 8 días

Estructura de red de tareas (PERT)

Introducción a la holgura

Cálculo de holgura.

Holgura = Duración ruta crítica − Duración del camino

Calculamos la holgura para las rutas **no críticas:**

Camino A → B → D: 8 − 7 = **1** día de holgura

Camino A → E → D: 8 − 6 = **2** días de holgura

Estructura de red de tareas (PERT)

Introducción a la holgura

Cálculo de holgura.

Interpretación:

Las tareas del camino A → C → D son críticas.

Las tareas de los otros caminos tienen margen de atraso.

Esto significa: pueden retrasarse un poco sin afectar el proyecto

Estructura de red de tareas (PERT)

Introducción a la holgura

Cálculo de holgura.

Ejemplo de atraso

Si la tarea B se retrasa 1 día:

Nuevo camino: 2 + (3+1) + 2 = 8 días

No afecta al proyecto

Estructura de red de tareas (PERT)

Introducción a la holgura

Cálculo de holgura.

Ejemplo de atraso (superando holgura)

Si la tarea B se retrasa 2 días:

Nuevo camino: 2 + (3+2) + 2 = 9 días

Ahora este camino pasa a ser el más largo.

El proyecto se retrasa

Estructura de red de tareas (PERT)

Introducción a la holgura

Conclusión

Las tareas críticas no tienen margen de error.

Las tareas no críticas tienen holgura

Pero: si se supera la holgura, el proyecto se retrasa

Estructura de red de tareas (PERT)

Introducción a la holgura

Idea clave

La holgura permite:

- absorber pequeños retrasos
- dar flexibilidad al proyecto

Pero no es infinita.

Gestionar la holgura es clave para evitar retrasos.

Actividad práctica: ejercicio 3.4 a 3.7

Ejercicio integrador Gantt + PERT: ejercicio 3.8

Introducción a las Metodologías de Desarrollo de Software

Qué es una Metodología/Modelo de Proceso?

- Es un marco de trabajo estructurado que define las actividades, acciones, tareas, hitos y productos de trabajo requeridos para construir software.

Enfoque Predictivo vs. Adaptativo

- **Orientadas a Planes / Prescriptivas (Tradicionales):** Priorizan la planificación detallada, la documentación y el orden secuencial antes de ejecutar el código. El progreso se mide en función del cumplimiento del plan inicial.
- **Agilidad:** Prioriza la adaptación al cambio, la entrega rápida de valor y la colaboración continua sobre la planificación rígida.

Metodologías Tradicionales principales (Visión General)

Las metodologías tradicionales se caracterizan por una estructura rígida y un control sistemático:

- **Modelo en Cascada (Waterfall):** Enfoque estrictamente secuencial donde cada fase (requisitos, diseño, código, pruebas) debe completarse antes de iniciar la siguiente.
- **Modelo en V (V-Model):** Variación directa de la cascada que asocia explícitamente cada fase de desarrollo con su correspondiente fase de verificación y pruebas.
- **Modelos Incrementales y Evolutivos (Espiral / Prototipado):** Enfoques donde el software se desarrolla mediante entregas o ciclos repetitivos pero bajo el marco de una planificación previa rigurosa.

Enfoque de Ian Sommerville

Modelos Orientados a Planes / *Plan-Driven*

Sommerville (2011) clasifica los procesos de desarrollo en modelos orientados a planes, definiéndolos como aquellos donde todas las actividades se planifican con antelación y el avance se mide contra dicho plan.

1. Modelo en Cascada (*Waterfall Model*)

- **Definición y Cita APA:**
  "El modelo en cascada es un ejemplo de un proceso orientado a planes en el que se deben planificar y programar todas las actividades del proceso antes de comenzar a trabajar en ellas. Es un enfoque secuencial donde cada actividad fundamental se representa como una fase separada" (Sommerville, 2011, p. 30).
- **Fases seguras:** Análisis de requisitos → Diseño del sistema y software → Implementación y prueba unitaria → Integración y prueba del sistema → Funcionamiento y mantenimiento.
- **Aplicabilidad:** Adecuado cuando los requisitos son bien conocidos y los cambios durante el desarrollo son mínimos o muy limitados.

Enfoque de Ian Sommerville

Modelos Orientados a Planes / *Plan-Driven*

2. Desarrollo Incremental (*Plan-Driven*)

- **Definición:**
  "El desarrollo incremental se basa en la idea de desarrollar una implementación inicial, exponiéndola a los comentarios del usuario y evolucionándola a través de varias versiones hasta que se desarrolla un sistema aceptable" (Sommerville, 2011, p. 32).
- **Perspectiva:** Puede ser tradicional (orientado a planes) si cada incremento se especifica y planifica en detalle antes de construirse.

Enfoque de Roger S. Pressman

Modelos de Proceso Prescriptivos

Pressman (2010) denomina a estas metodologías **Modelos Prescriptivos**, los cuales imponen una disciplina estructurada y un flujo de trabajo ordenado sobre la ingeniería de software.

1. Modelo en Cascada / Ciclo de Vida Clásico

- **Definición:**
  "El modelo en cascada, a veces llamado ciclo de vida clásico, sugiere un enfoque sistemático y secuencial para el desarrollo del software, que comienza con el especificación de requisitos por parte del cliente y avanza a través de la planeación, el modelado, la construcción y el despliegue" (Pressman, 2010, p. 39).
- **Fases según Pressman:** Comunicación → Planeación → Modelado → Construcción → Despliegue.

Enfoque de Roger S. Pressman

Modelos de Proceso Prescriptivos

2. Modelo en V (Variación del Ciclo de Vida)

- **Definición y Cita APA:**
  "El modelo en V proporciona una forma de visualizar cómo se relacionan las acciones de verificación con las acciones de ingeniería primarias. Muestra que la calidad del software se valida en cada paso de desarrollo" (Pressman, 2010, p. 41).

3. Modelos de Proceso Evolutivo (El Modelo en Espiral)

- **Definición y Cita APA:**
  "El modelo en espiral es un generador de modelos de proceso guiado por el riesgo, que se adapta para adaptar el software a través de iteraciones evolutivas. Combina la naturaleza iterativa del prototipado con los aspectos controlados y sistemáticos del modelo en cascada" (Pressman, 2010, p. 45).

Tabla Comparativa (Sommerville vs. Pressman)

Criterio	Ian Sommerville	Roger S. Pressman
Categorización General	**Procesos orientados a planes (*Plan-Driven*)**: Todas las actividades se planifican rigurosamente antes de ejecutar.	**Modelos Prescriptivos**: Marcos de trabajo que prescriben un conjunto ordenado de actividades y entregables.
Modelo en Cascada	Enfoque lineal rígido. Ideal solo para sistemas complejos y distribuidos con requisitos estables.	Conocido como "Ciclo de vida clásico". Destaca su flujo secuencial desde la comunicación hasta el despliegue.
Modelo en V	Lo considera una variación operacional de la cascada para relacionar especificaciones con validación.	Lo formaliza como la extensión visual directa para asociar las pruebas tempranas con las decisiones de diseño.
Modelo en Espiral	Trata la espiral como un marco general iterativo centrado en la **gestión de riesgos**.	Lo clasifica explícitamente dentro de la categoría de **Modelos Evolutivos Prescriptivos**.

Criterios para Fundamentar la Elección del Modelo

1. Si se fundamenta en Ian Sommerville (*Plan-driven vs. Agile*):

El estudiante debe evaluar su proyecto bajo el criterio de **estabilidad e incertidumbre**:

- **Elige Cascada si:** Los requisitos son completamente conocidos, estables y el proyecto se desarrolla en un entorno distribuido o con múltiples equipos que requieren especificaciones formales antes de construir (Sommerville, 2011).
- **Elige Incremental si:** Requiere un equilibrio entre planificar el marco general (*plan-driven*) y entregar versiones parciales para obtener retroalimentación sin comprometer la estructura arquitectónica inicial (Sommerville, 2011).
- **Elige Híbrido si:** El sistema combina un núcleo de alta especificidad (que requiere planificación previa en **Cascada**) con subsistemas de usuario que deben adaptarse iterativamente mediante entregas **Incrementales** (Sommerville, 2011).

Criterios para Fundamentar la Elección del Modelo

2. Si se fundamenta en Roger Pressman (*Modelos Prescriptivos*):

El estudiante debe evaluar su proyecto bajo el criterio de **naturaleza del problema y gestión de riesgos**:

- **Elige Cascada / Ciclo de vida clásico si:** El problema está bien definido, los requisitos no van a sufrir variaciones y el flujo de trabajo requiere una secuencia lineal estricta (Pressman, 2010).
- **Elige Modelo en V si:** La prioridad principal del sistema es la alta confiabilidad o calidad, exigiendo que cada decisión de diseño tenga una prueba de validación directamente acoplada (Pressman, 2010).
- **Elige Espiral / Evolutivo si:** El software es complejo, de gran escala, o los requisitos son ambiguos al inicio, lo que exige un análisis riguroso de riesgos en cada iteración antes de proceder a la construcción (Pressman, 2010).
- **Elige Híbrido si:** La heterogeneidad del problema requiere adaptar el marco de trabajo (*framework adaptation*), combinando modelos prescriptivos como el **Modelo en V** para componentes críticos (ej. seguridad o pagos) y **Modelos Evolutivos** para módulos de interfaz dinámica (Pressman, 2010).

Criterios para Fundamentar la Elección del Modelo

Consigna Obligatoria:

- **Selección de Modelos: No** se debe seleccionar únicamente el modelo en Cascada. Es requisito integrarlo con **otro modelo** de desarrollo (por ejemplo: Incremental, Modelo en V o Espiral) constituyendo un enfoque híbrido.
- **Fundamentación:** El estudiante debe explicar explícitamente cómo las características y necesidades de su proyecto se adecúan a la combinación de ambos modelos.
- **Uso de Fuentes:** Los modelos seleccionados pueden sustentarse en la teoría de **Sommerville**, de **Pressman** o de ambos. Es indispensable incluir la **cita bibliográfica correspondiente en formato APA** para cada autor utilizado.

Enfoques de la organización del trabajo cooperativo y colaborativo

Trabajo en equipo

“El desarrollo de software es una actividad inherentemente social que requiere la coordinación de múltiples personas con diferentes habilidades.” - Sommerville (2016)

No es una actividad individual. Requiere:

- Comunicación constante
- Coordinación de tareas
- Integración de resultados

Enfoques de la organización del trabajo cooperativo y colaborativo

Enfoques de organización del trabajo

“La estructura del equipo y la forma en que se organiza el trabajo influyen directamente en la calidad y productividad del software desarrollado.” - Pressman (2010)

Existen distintas formas de organizar el trabajo:

- Cooperativo
- Colaborativo

Cada enfoque responde a distintos contextos de proyecto

Enfoques de la organización del trabajo cooperativo y colaborativo

Trabajo cooperativo

“En equipos organizados de manera tradicional, el trabajo se divide en partes bien definidas que son desarrolladas de forma individual antes de su integración.” - Pressman (2010)

- División clara de tareas
- Baja interacción durante el desarrollo
- Integración al final

Enfoques de la organización del trabajo cooperativo y colaborativo

Trabajo cooperativo

Ejemplo en software - Sistema de gestión:

- Desarrollador 1 → Interfaz
- Desarrollador 2 → Lógica de negocio
- Desarrollador 3 → Base de datos

Resultado:

- Cada módulo se desarrolla por separado
- Se integran al final del proyecto

Enfoques de la organización del trabajo cooperativo y colaborativo

Trabajo colaborativo

“Los enfoques modernos de desarrollo promueven equipos autoorganizados donde los miembros trabajan conjuntamente y comparten responsabilidades.”

- Sommerville (2016)

- Trabajo conjunto sobre las mismas tareas
- Comunicación constante
- Responsabilidad compartida

Enfoques de la organización del trabajo cooperativo y colaborativo

Trabajo colaborativo

Ejemplo en software - Equipo ágil:

- Daily meetings
- Revisión conjunta del código
- Programación en pares
- Decisiones en equipo

Resultado:

- Producto construido de forma incremental
- Ajustes continuos

Enfoques de la organización del trabajo cooperativo y colaborativo

Diferencias clave

Aspecto	Cooperativo	Colaborativo
Organización	División de tareas	Trabajo conjunto
Interacción	Baja	Alta
Responsabilidad	Individual	Compartida
Integración	Al final	Continua

Enfoques de la organización del trabajo cooperativo y colaborativo

Relación con metodologías

Enfoque cooperativo:

- Modelos tradicionales
- Fases separadas

Criterio de selección →

- Equipos grandes
- tareas bien definidas

Enfoque colaborativo:

- Metodologías ágiles
- Iteración e incremento

Criterio de selección →

- Proyectos cambiantes
- Equipos pequeños o medianos

Enfoques de la organización del trabajo cooperativo y colaborativo

“La efectividad de un equipo de software depende tanto de la organización del trabajo como de las habilidades técnicas.” - Pressman (2010)

<u>En la práctica:</u>

- Se combinan ambos enfoques
- Se adaptan al contexto del proyecto

Proceso de gestión de cambios

Definición

“La gestión de cambios es el proceso de identificar, analizar, aprobar e implementar modificaciones en un sistema de software.” - Sommerville (2016)

Los sistemas cambian constantemente.

Los cambios deben:

- Ser controlados
- Ser evaluados
- Mantener la calidad del sistema

Proceso de gestión de cambios

Importancia de la gestión de cambios

“El cambio es inevitable en todos los proyectos de software y debe ser gestionado de manera sistemática para evitar efectos negativos en el producto.” - Pressman (2010)

- Requerimientos cambian
- Errores deben corregirse
- Nuevas funcionalidades aparecen

Sin control → caos en el proyecto

Proceso de gestión de cambios

A partir de la definición anterior, la gestión de cambios se implementa mediante un proceso estructurado:

Etapas:

- Solicitud de cambio
- Análisis de impacto
- Aprobación o rechazo
- Implementación
- Verificación



- Permite controlar cada modificación
- Reduce riesgos en el sistema
- Asegura trazabilidad de los cambios

Proceso de gestión de cambios

Ejemplo de cambio en software - Sistema de ventas:

Cliente solicita: Agregar descuentos por usuario

Proceso:

- Se analiza impacto
- Se aprueba
- Se implementa
- Se prueba

Proceso de gestión de cambios

Seguimiento y control de cambios

“El seguimiento de cambios permite asegurar que todas las modificaciones sean registradas, controladas y trazables.” - Pressman (2010)

- Saber qué cambió
- Quién lo cambió
- Cuándo se cambió
- Por qué se cambió

Proceso de gestión de cambios

Control de versiones

“El control de versiones es una actividad de gestión de configuración que permite gestionar múltiples versiones de un sistema de software.” - Sommerville (2016)

Permite:

- Guardar historial
- Recuperar versiones anteriores
- Trabajar en equipo